Nabbihni | Product Expiry Tracker

Use Cases and UML Lab
Effat University - CS3151 Software Engineering - Section 1

Team: Dana Eshmawi, Zainab Tantawi, Wajd Almadani
Instructor: Dr. Passent Elkafrawy
Date: 11 October 2026

System scope

A bilingual mobile app that stores manually entered product expiry dates, organizes products by category, displays expiry status, and schedules local reminders. The prototype uses one local product collection on the device. Login, cloud sharing, barcode scanning, payments, and advertisements are outside this lab scope.

Contents

D1 Actors, prioritized use cases and traceability

D2 Brief specifications and two fully dressed use cases

D3 Use case diagram

D4 Activity diagram: Edit Product

D5 Sequence diagram: Add Product

D6 Domain model / class diagram

D7 Consistency checks and peer-review sheet

Modeling assumptions

The phone notification service is outside the app boundary. Notification permission is controlled by the phone. The collection represents the local user context; no account is required. Proposed rules below make the narratives testable and should be confirmed by the team.

An expiry date before today means expired; today means expires today. Upcoming means today through seven days ahead, inclusive. Reminder lead time is 1, 3 or 7 days, at 09:00 local time. Only future reminder times are scheduled. A missed reminder time produces an in-app message, not an invented past notification.

D1 | Actors and use case priorities

Actor	Description / persona mapping	Goals and constraints
App User	Layan: student; Abeer: household shopper; Rawan: beauty-product user. All share the same role.	Manage product records, organize products, check expiry, and use Arabic or English. Dates must be entered accurately.
Phone Notification Service	External phone service used for local reminders.	Report permission and schedule/cancel reminders. Delivery timing remains subject to phone settings.

ID	Use case	Actor	Priority
UC-1	Add Product	App User	M
UC-2	Edit Product	App User	M
UC-3	Delete Product	App User	M
UC-4	Browse Products	App User	M
UC-5	View Upcoming Expiry	App User	M
UC-6	Search / Filter Products	App User	S
UC-7	Change Language	App User	M
UC-8	Configure Reminder	App User	M

M = Must have; S = Should have. Notification Service supports UC-8. UC-1 and UC-2 may optionally invoke UC-8 while editing a product. UC-3 cancellation uses the external service internally. No actor generalization is needed because all personas have the same permissions.

Deferred (Won't for now): barcode scanning, shared cloud accounts, subscriptions and retailer integration. They are excluded from the use case diagram.

D1 | Requirements and traceability

R1-R8 below consolidate the existing user stories and feasibility scope. They are lab modeling identifiers, not replacements for the original document's REQ numbering.

Req	Requirement / source	Use cases
R1	Save product name, category and valid expiry date. Source: earlier Lab 3 REQ-6; Layan scenario.	UC-1
R2	Edit product details and persist changes. Source: feasibility product records and acceptance checks.	UC-2
R3	Delete product and cancel its reminder. Source: Rawan scenario; feasibility acceptance checks.	UC-3
R4	Organize products by category, including separate makeup and skincare. Source: earlier Lab 3 REQ-3 / REQ-5; Abeer and Rawan.	UC-1, UC-2, UC-4, UC-6
R5	Show upcoming products in nearest-expiry order and identify expired items. Source: earlier Lab 3 REQ-4; feasibility expiry logic.	UC-4, UC-5
R6	Search names and filter by category. Source: Rawan persona feature; feasibility functions.	UC-6
R7	Switch core screens between Arabic and English. Source: earlier Lab 3 REQ-1; Layan persona.	UC-7
R8	Schedule reminders; show permission status; update or cancel reminders when products change. Source: earlier Lab 3 REQ-2; feasibility notifications.	UC-1, UC-2, UC-3, UC-8

Proposed measurable constraints

N1: After successful saving, records and language preference remain after restarting the app. N2: All eight core use cases have Arabic and English text; Arabic screens use right-to-left layout. N3: Lists show results within two seconds for a collection of up to 100 products on the chosen test phone. N4: Saving works without internet. N5: A failed or denied reminder request is shown explicitly; the app must not claim it was scheduled.

D2 | Brief use case specifications

Use case	Preconditions	Success result / main flow
UC-1 Add Product	App open; categories available.	Enter details; validate; save; optionally configure reminder. Product appears in its category.
UC-2 Edit Product	A saved product exists.	Select product; change fields; save; cancel old reminder; optionally schedule a new one.
UC-3 Delete Product	A saved product exists.	Select product; confirm deletion; cancel reminder; remove record.
UC-4 Browse Products	App open.	Open collection; calculate status; display products by category. Empty collection shows a message.
UC-5 View Upcoming Expiry	App open.	Calculate remaining days; select dates from today to +7 days; sort nearest first.
UC-6 Search / Filter Products	App open.	Enter name or select category; show matching products or no-results message.
UC-7 Change Language	App open.	Select Arabic or English; save preference; update text and layout.
UC-8 Configure Reminder	A saved product exists.	Choose enabled state and lead time; validate schedule time; check permission; schedule or cancel.

Common failure guarantee: invalid input does not create a product or overwrite an existing one. Cancellation before saving leaves the record unchanged. Persistence errors are reported to the user. Reminder failures do not hide the saved product.

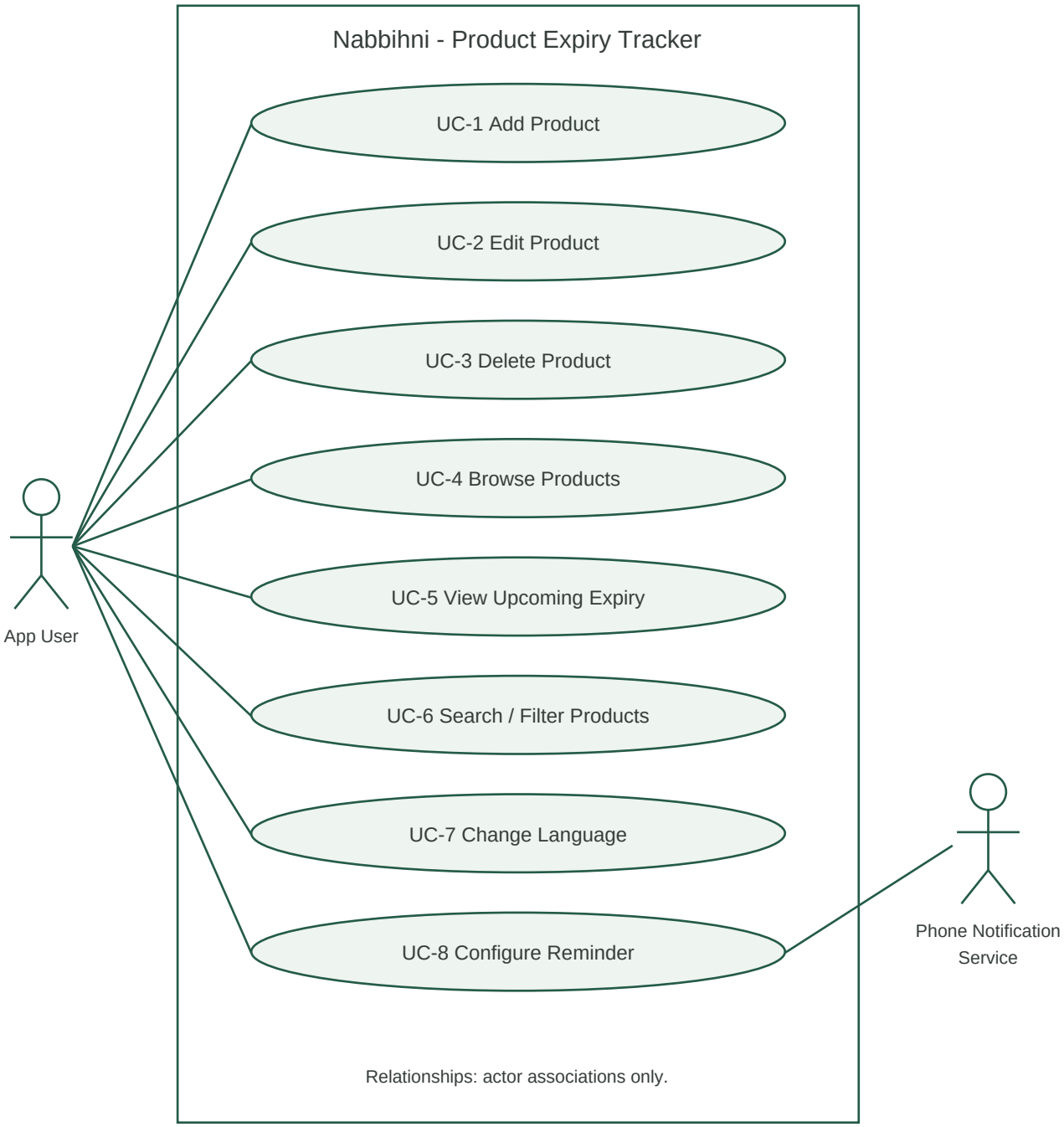
D2 | UC-1 Add Product

Field	Specification
Primary actor / trigger	App User selects Add Product.
Stakeholders	User wants accurate expiry tracking; Notification Service needs a valid future request and permission.
Preconditions	App is open. Supported categories are available. No login or internet connection is required.
Success guarantee	A new Product belongs to the ProductCollection and one Category. If a reminder was selected and accepted, its schedule and status are stored.
Minimum guarantee	No incomplete product is saved. A rejected reminder is never shown as active.
Main success scenario	1. User enters name, category, expiry date and selects a reminder lead time. 2. App validates required fields and date. 3. App saves the product locally and receives its saved identifier. 4. App calculates a future reminder time. 5. App checks notification permission; permission is granted. 6. App asks Notification Service to schedule the reminder; service accepts. 7. App records reminder status and displays the saved product and confirmation.
Extensions	2a. Missing name, unknown category or invalid date: show field error; return to step 1. 2b. Date is today or in the past: allow recording, label status correctly, and skip advance reminder. 3a. Saving fails: show error; create no reminder; user retries or cancels. 4a. Reminder disabled: skip steps 4-6; save without an active reminder. 4b. Calculated time has passed: skip scheduling; show reminder unavailable and current expiry status. 5a. Permission denied: retain product; show reminder inactive and guidance to phone settings. 6a. Service rejects schedule: retain product; mark reminder failed; offer retry through UC-8.
Special requirements	N1, N2, N4 and N5 apply. Required fields: nonblank name, supported category, valid calendar date.
Constraints / open issues	No automatic safety judgment. Confirm reminder times and supported phone platform before implementation.

D2 | UC-2 Edit Product

Field	Specification
Primary actor / trigger	App User selects an existing product to edit its expiry date.
Stakeholders	User needs corrected information; Notification Service must not retain an outdated reminder.
Preconditions	Product exists in the local collection. Its current details and reminder state can be retrieved.
Success guarantee	Updated fields are stored; status is recalculated; the previous reminder is cancelled and a replacement is accepted when eligible.
Minimum guarantee	If validation or saving fails, the old record and reminder remain. If notification operations fail after saving, the saved record remains with a visible failure status.
Main success scenario	1. User opens a product and changes its expiry date and reminder preference. 2. App validates the changed fields. 3. App saves updated product details. 4. App cancels the previous reminder, if any. 5. App checks whether the selected reminder is enabled, in the future and permitted. 6. Notification Service accepts a replacement reminder. 7. App stores reminder status and displays updated product and expiry status.
Extensions	2a. Invalid fields: show error and return to step 1. 3a. Saving fails: preserve original record and original reminder; report failure. 4a. Cancellation fails: do not schedule a replacement; mark notification update failed and offer retry. Warn that an old alert may still appear. 5a. Reminder disabled, expired/today date, past reminder time or permission denied: keep updated product, leave no active reminder after successful cancellation, and explain why. 6a. Scheduling fails: keep updated product; mark reminder failed; offer retry through UC-8. 1a. User cancels before saving: end with no change.
Special requirements	N1, N2, N4 and N5 apply. Reuse product identity when replacing a reminder to avoid duplicate alerts.
Constraints / open issues	Save the record before changing the reminder. Confirm cancellation behavior on the supported phone platform.

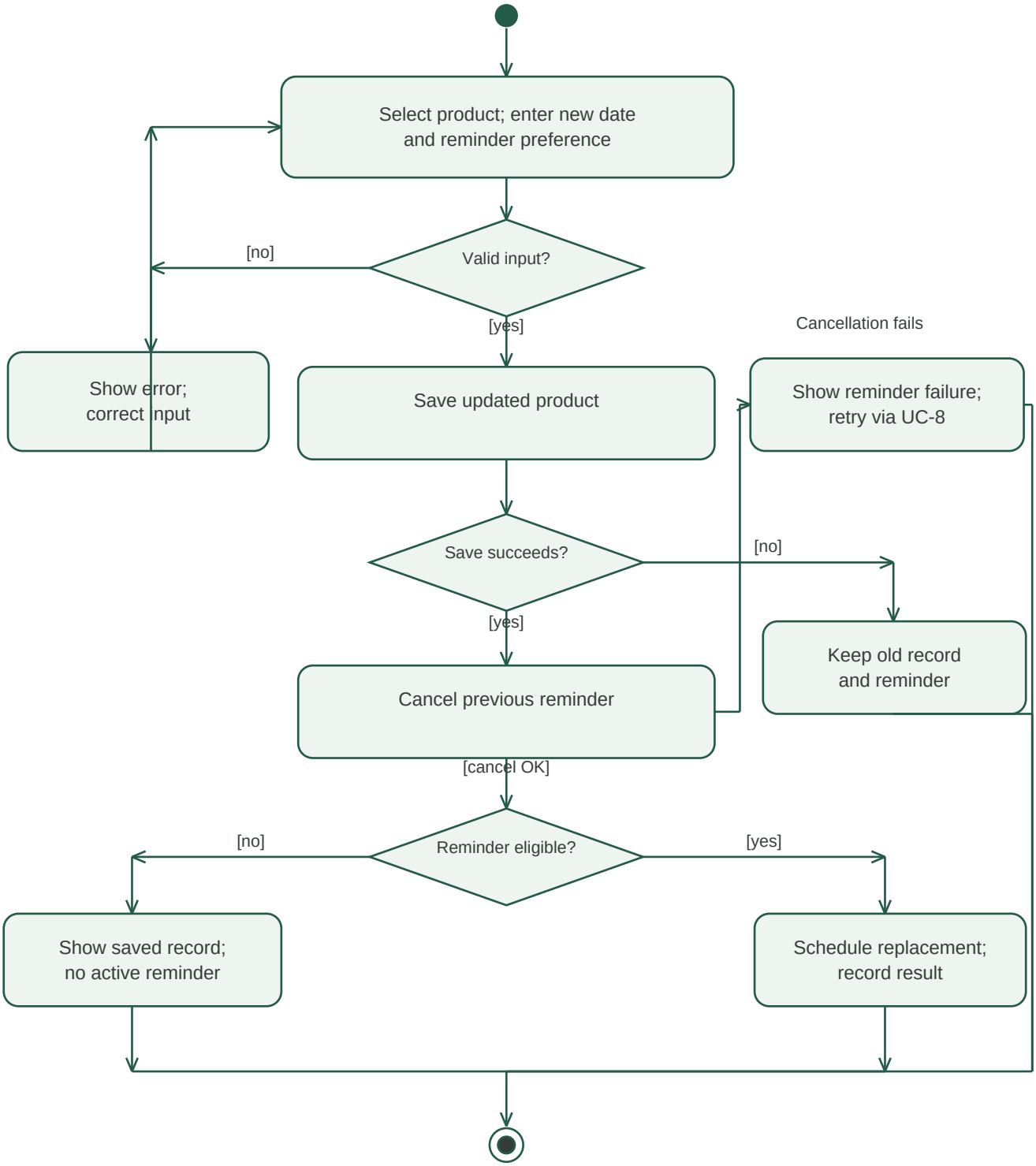
D3 | Use case diagram



Each oval is a user goal within the app boundary. Associations do not show workflow order. Include, extend and generalization are omitted because no mandatory reusable goal or distinct actor permissions justify them in this scope.

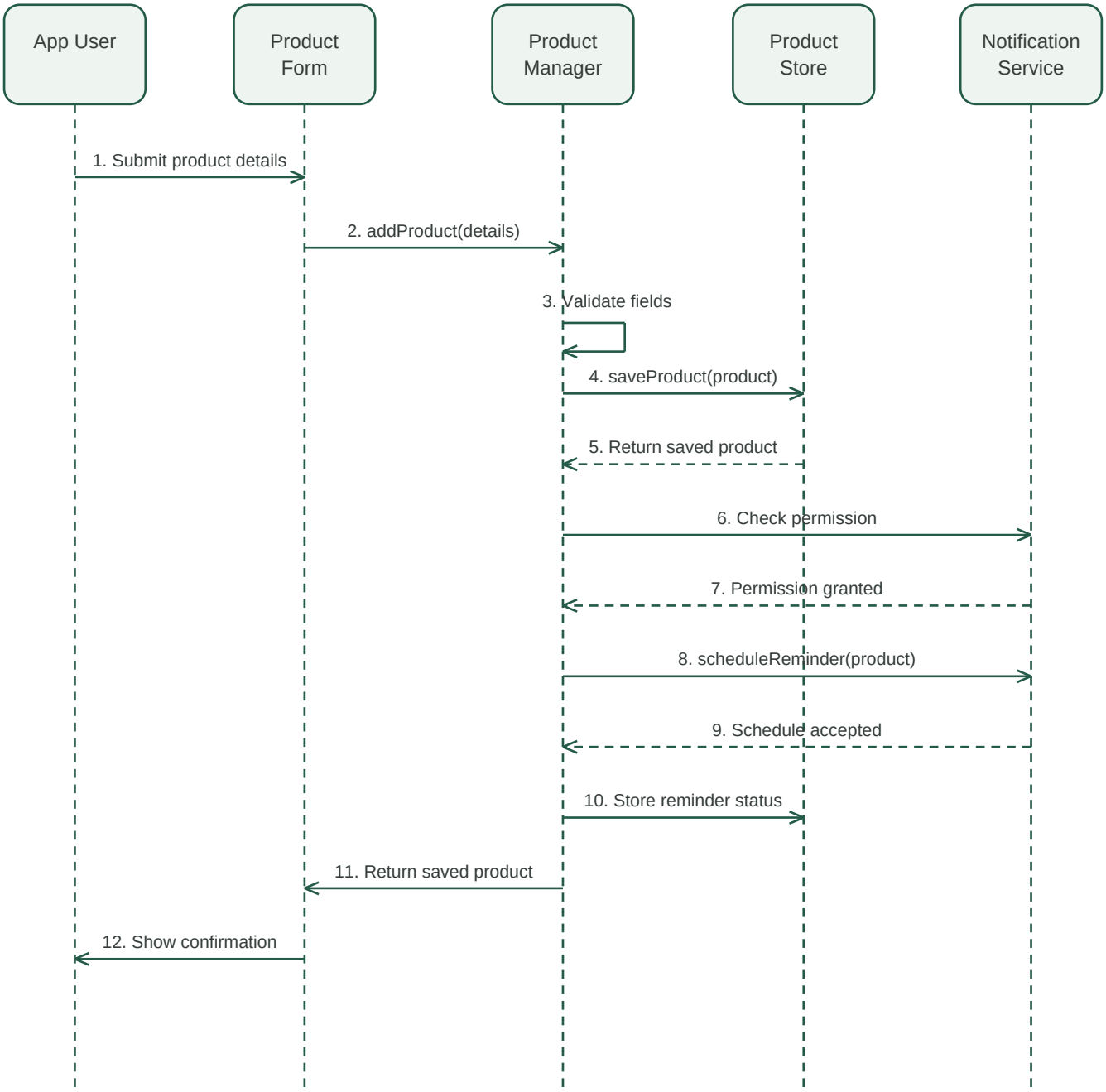
Reminder configuration is an optional part of Add Product or Edit Product and can also be performed separately. Phone Notification Service supports scheduling and cancellation; it is not the app itself.

D4 | Activity diagram: Edit Product



Eligible = enabled AND future schedule time AND notification permission granted. A scheduling or cancellation failure follows the failure branch in UC-2: retain the saved product, show a visible failure and offer retry. Actions are sequential; no independent tasks justify a fork/join.

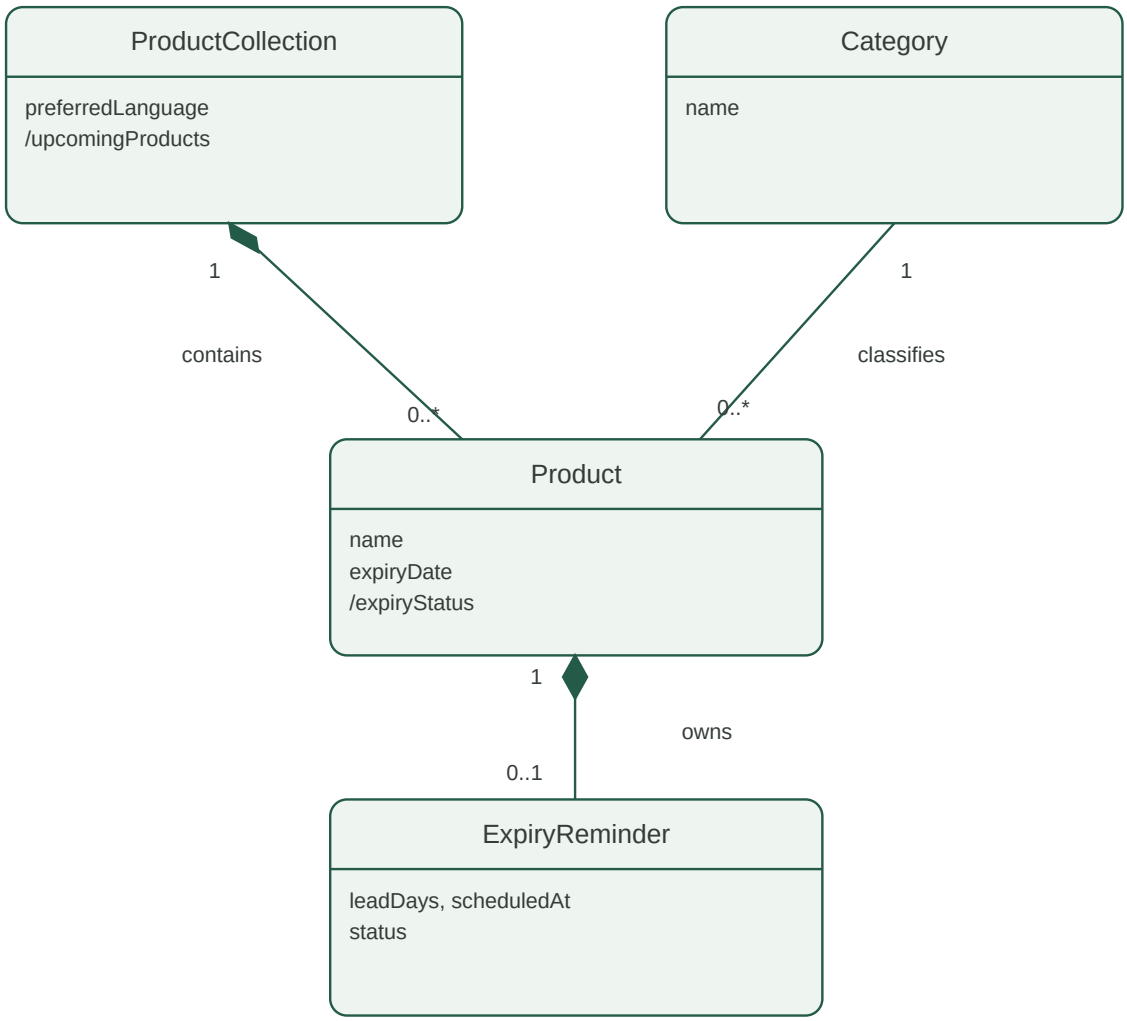
D5 | Sequence diagram: Add Product



Main success scenario: valid future date, future reminder time, granted permission and accepted schedule. Product Manager calculates the reminder time during validation. Solid arrows are requests; dashed arrows return saved data or service results. Form and service participants are design components; they are excluded from the domain model.

N4 note: Product Store is local, so product saving requires no internet. N5 note: confirmation of an active reminder occurs only after the phone service accepts it.

D6 | Domain model / class diagram



Concept	Meaning
ProductCollection	The device-local collection and preferred language. Owns zero or more products.
Product	A tracked item with a name and entered expiry date. /expiryStatus is derived from the current date.
Category	A classification such as food, medicine, supplements, makeup, skincare, hair or household. Each product has exactly one category.
ExpiryReminder	Optional reminder configuration and outcome owned by one product. Includes selected lead days, schedule time and status.

Filled diamonds indicate composition: deleting a product removes its reminder; the collection owns its products. A category may classify zero or more products and remains available when a product is deleted. Multiplicities: collection 1 to products 0..*; category 1 to products 0..*; product 1 to reminder 0..1. No database tables, UI classes or notification API classes are included.

D7 | Consistency and review

Check	Evidence / result
Actor coverage	All three personas map to App User; external Notification Service is represented.
Use case coverage	UC-1 through UC-8 each have App User association and appear in D3.
Traceability	R1-R8 map to use cases in D1; persona and feasibility sources are named.
Sequence consistency	D5 follows UC-1: validate, save, check permission, schedule, record status, confirm.
Activity consistency	D4 follows UC-2 input validation, save failure, reminder replacement and eligibility branches. Cancellation/scheduling failures are clarified in its note.
Domain consistency	Collection, Product and Category appear in product management narratives; ExpiryReminder appears in UC-1, UC-2 and UC-8.
Constraints consistency	Persistence and offline saving are noted in D5; reminder eligibility guards appear in D4. Bilingual text applies to all narratives.
Notation and scope	Actors remain outside boundary; composition and multiplicities are stated; no unjustified include/extend relations.

Peer-review sheet - pending another team

Reviewer team: _____ Reviewer names: _____
Date: _____ Version reviewed: _____

[ ] Pass [ ] Revise - Actors cover personas and external systems

[ ] Pass [ ] Revise - Use case names are clear user goals

[ ] Pass [ ] Revise - Include / extend relationships are appropriate or unnecessary

[ ] Pass [ ] Revise - Activity and sequence diagrams match the narratives

[ ] Pass [ ] Revise - Domain concepts and multiplicities make sense

[ ] Pass [ ] Revise - Preconditions and success guarantees are clear

[ ] Pass [ ] Revise - All artifacts are consistent

[ ] Pass [ ] Revise - Layout and labels are clear

Reviewer comments / requested changes:

_____

_____

Team response and changes made:

_____

Sources and submission notes

Prepared from the uploaded documents below. The Movie Shop examples in the earlier lab are excluded from the Nabbihni model.

Lab3_ From Personas & Requirements To Use Cases And Uml.docx - required D1-D7 deliverables, templates, consistency and peer-review criteria.

CS3151_Lab2_Persona_Solved.docx - Layan, Abeer and Rawan; bilingual interface, categories, reminders, upcoming list and search/filter needs.

CS3151_Lab3.docx - project user stories and REQ-1 through REQ-6; the Movie Shop exercises are not project scope.

Product_Expiry_Tracker_Feasibility_Study.docx - manual entry, local storage, editing/deletion, notification permission, persistence and prototype scope.

Proposed decisions to confirm

Seven-day upcoming window; lead time choices of 1, 3 or 7 days; 09:00 local reminder time; two-second list target for 100 products; support for saving already expired products. These are explicit modeling assumptions, not claimed survey findings or implemented features.

Before submission

Ask another team to complete the peer-review sheet and incorporate their feedback. Replace this PDF with the reviewed version. The requested naming convention is TeamName_UML_Lab.pdf; this file uses Nabbihni as the team/project name. No PlantUML is used, so no PlantUML source ZIP is required.